



PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG THÔNG TIN

Chương 7: Lựa chọn công nghệ

ThS. Nguyễn Văn Hữu Hoàng

I NỘI DUNG

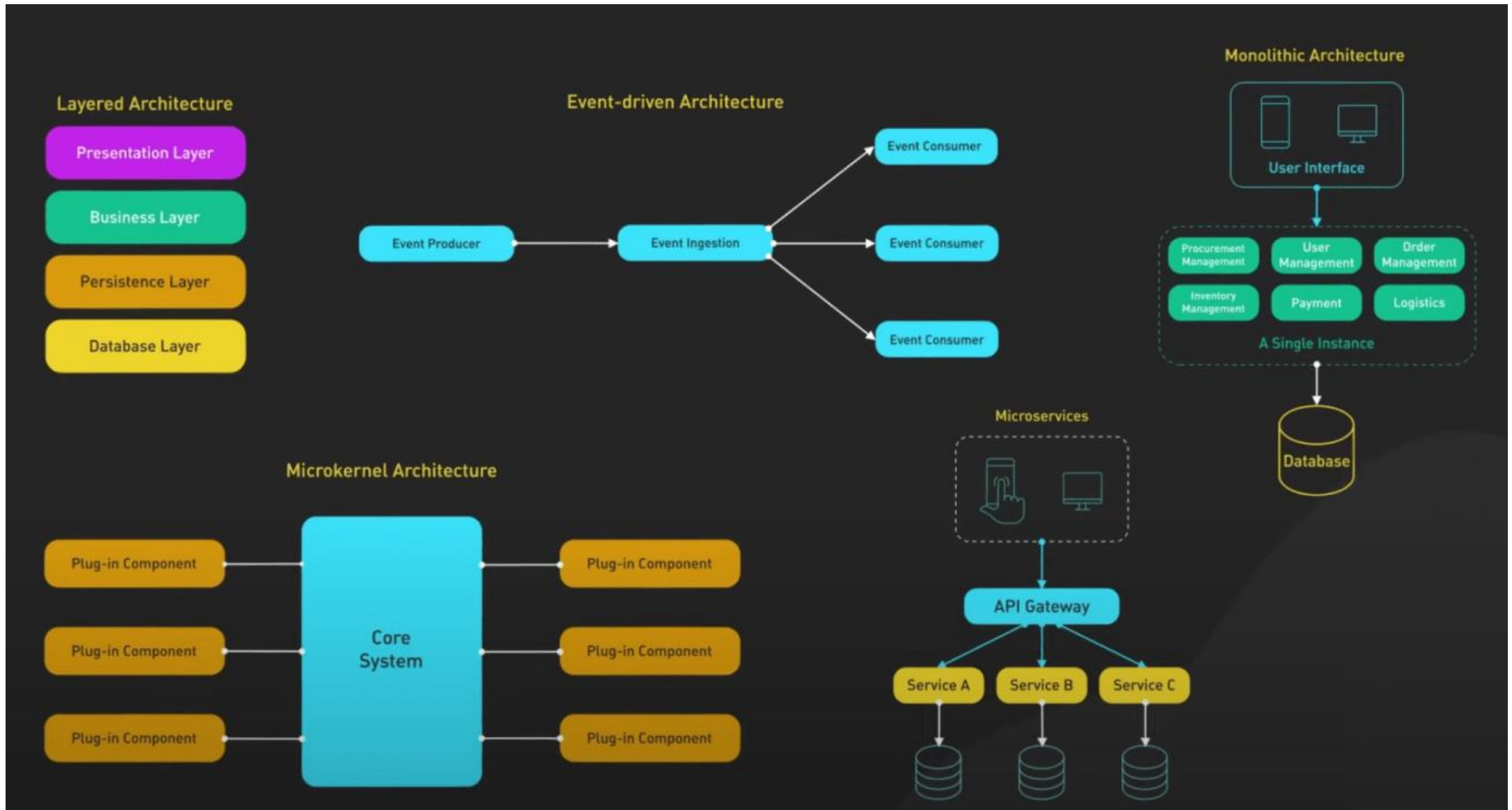
Chương này tập trung trình bày một số chủ đề sau đây:

- 7.1. Giới thiệu
- 7.2. Các mô hình kiến trúc phần mềm
- 7.3. Mô hình MVC
- 7.4. Công nghệ lập trình Front End và Back End
- 7.5. Lựa chọn cơ sở dữ liệu
- 7.6. Lựa chọn công nghệ lưu trữ
- 7.7. Các công nghệ khác

| 7.1 GIỚI THIỆU

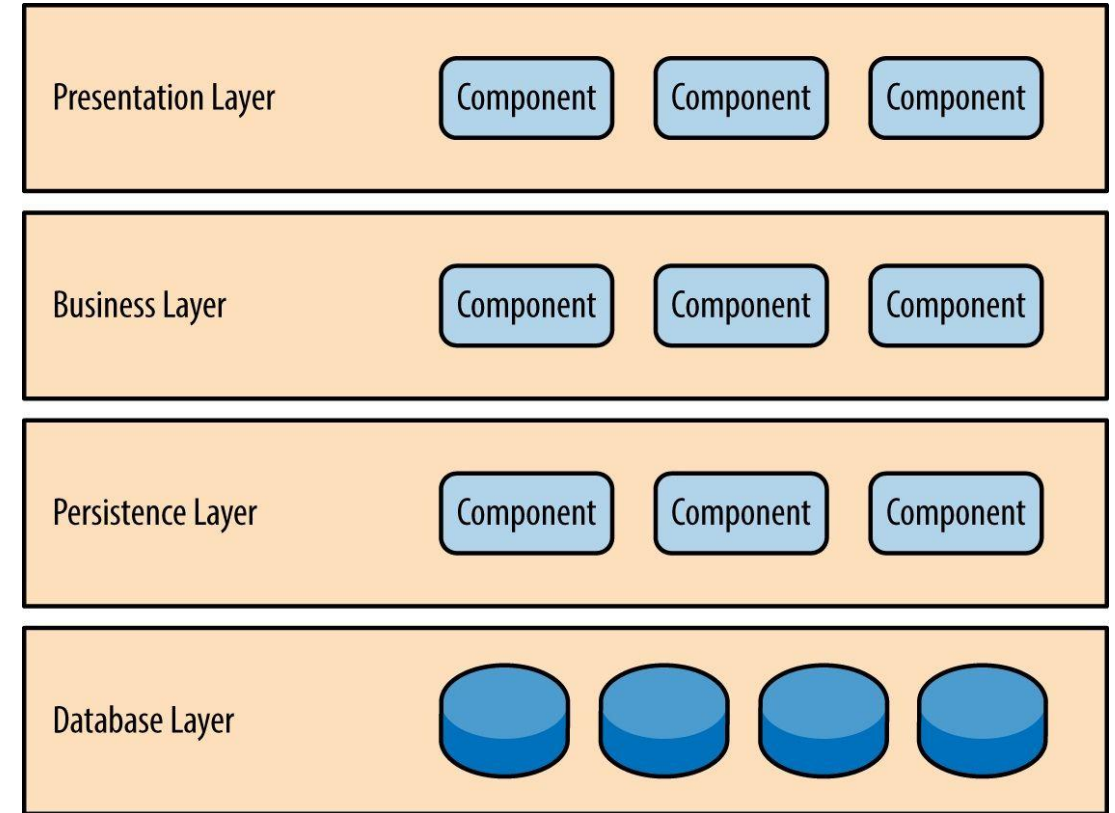
- Mặc dù chúng ta có bộ tài liệu yêu cầu và phân tích đầy đủ nhưng vẫn **chưa đưa ra lựa chọn nào về công nghệ cài đặt** mà chúng ta sẽ sử dụng.
- Giai đoạn lựa chọn công nghệ có thể kết hợp với bước **Thiết kế chi tiết**.
- Việc lựa chọn công nghệ có vai trò vô cùng quan trọng trong một quy trình sản xuất phần mềm.
- Lựa chọn công nghệ phải:
 - Giải quyết được yêu cầu của bài toán.
 - Đáp ứng các yêu cầu phi chức năng.
 - Theo yêu cầu của bên đặt hàng sản phẩm (nếu có).
 - Cần chọn mô hình kiến trúc phần mềm trước khi chọn công nghệ phát triển.
- Không phải công nghệ mới nhất là tốt nhất mà quan trọng là **sự phù hợp**.

| 7.2 CÁC MÔ HÌNH KIẾN TRÚC PHẦN MỀM

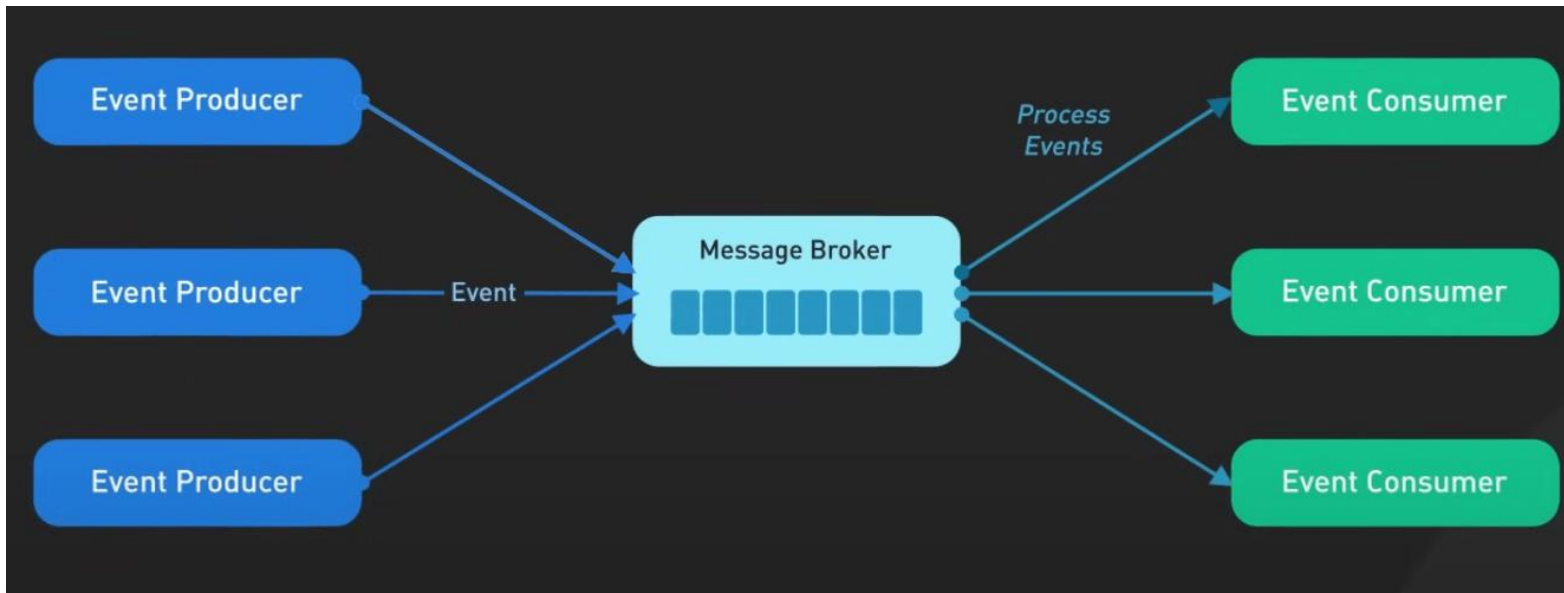


I KIẾN TRÚC PHÂN LỚP

- Kiến trúc phân lớp (Layered architecture - còn được gọi là N-tier architecture) là kiến trúc phổ biến từ những năm 90 cho tới ngày nay. Mỗi lớp trong kiến trúc này có chức năng riêng và tương tác với lớp ngay trên hoặc dưới nó. Các mô hình sử dụng kiến trúc này bao gồm MVC, MVVM, ...
- Các thành phần (component) trong kiến trúc phân lớp được tổ chức thành các lớp logic ngang, mỗi lớp thực hiện một vai trò cụ thể trong ứng dụng (ví dụ: logic trình bày - User interface hoặc logic nghiệp vụ - Business).
- Mặc dù không có con số cụ thể về số lượng và loại lớp phải có, kiến trúc phân lớp thường bao gồm bốn lớp tiêu chuẩn: **presentation**, **business**, **persistence**, và **database**.

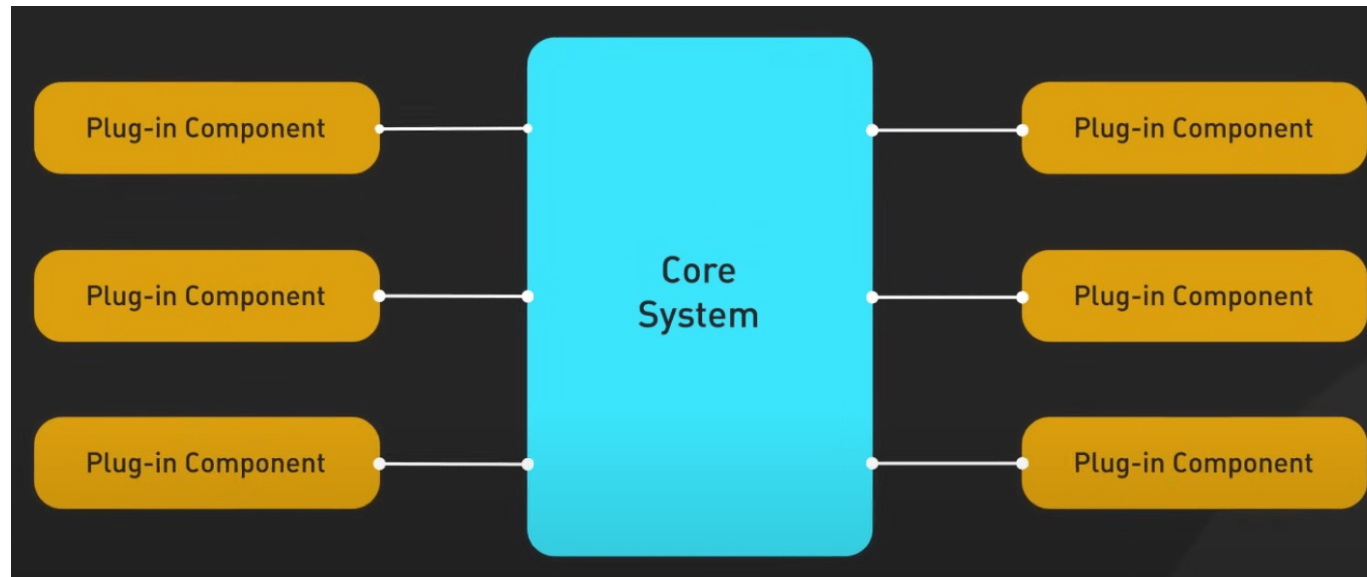


| KIẾN TRÚC HƯỚNG SỰ KIỆN



- **Kiến trúc hướng sự kiện** (Event-driven architecture - EDA) là một mô hình kiến trúc phần mềm trong đó các thành phần hoặc dịch vụ của hệ thống giao tiếp với nhau chủ yếu thông qua việc sản xuất và tiêu thụ các sự kiện.
- Các "sự kiện" (event) này có thể là các hành động người dùng, cập nhật dữ liệu, hoặc các thông báo từ các hệ thống khác. EDA giúp tạo ra các hệ thống linh hoạt, mở rộng và dễ tích hợp.

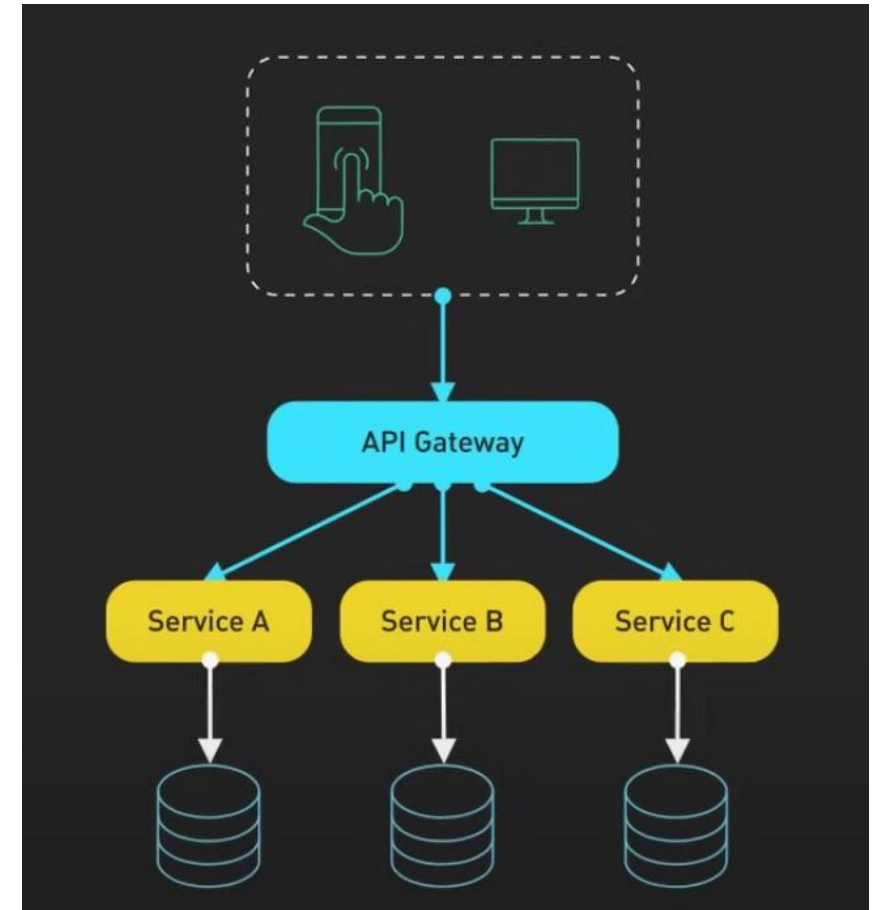
| KIẾN TRÚC VI NHÂN



- **Kiến trúc vi nhân** (Microkernel architecture), còn được gọi là Kiến trúc plug-in, là một mẫu kiến trúc phần mềm, trong đó, hệ thống được phân chia thành các module hoặc các thành phần nhỏ và độc lập (microkernel), chịu trách nhiệm cho các nhiệm vụ riêng biệt như xử lý dữ liệu, giao tiếp, lưu trữ, ...
- Microkernel lấy cảm hứng từ kernel của hệ điều hành. Ý tưởng cốt lõi là hệ thống có một phần được gọi là core system (microkernel) thực hiện các chức năng quan trọng nhất, trong khi, để cho plug-in thực hiện các chức năng khác, dễ dàng tháo lắp theo yêu cầu của business.

I KIẾN TRÚC MICROSERVICE

- **Microservice** vốn là một thuật ngữ được đặt ra vào đầu những năm 2010, đề cập đến phong cách phát triển phần mềm trong đó các ứng dụng bao gồm các dịch vụ nhỏ và độc lập, thường gọi là "microservice".
- Mỗi microservice được thiết kế để thực hiện một chức năng duy nhất hoặc một nhóm nhỏ các chức năng liên quan, được quản lý độc lập với nhau.
- Các thành phần của kiến trúc microservice cơ bản bao gồm:
 - **Service**: chứa logic nghiệp vụ của microservice, chỉ có service mới được truy cập vào database
 - **Database**: một tập hợp các bảng chứa dữ liệu phục vụ cho logic nghiệp vụ của microservice.
 - **API gateway**: cung cấp một điểm truy cập thống nhất cho các microservice, giúp quản lý, điều phối và bảo mật giao tiếp giữa các service với nhau trong kiến trúc microservices.



Maturity model for .NET applications modernization



Existing apps

.NET web apps (on-premises)

Monolithic or N-Tier
architectures



On-premises

Cloud Infrastructure-Ready

Monolithic or N-Tier
architectures



IaaS
(Infrastructure as a Service)



Azure

Cloud-Optimized PaaS

Monolithic or N-Tier
architectures



Managed services



And
Windows Containers



Azure

Cloud-Native

PaaS for microservices and serverless

Microservices and serverless architectures



PaaS for containerized microservices
+ Serverless computing
+ Managed services



Azure

Base Cloud Environment and cross-cutting concerns: Network, Hybrid-cloud, Identity/Auth, Cost control and Operations model

Migrate / Rehost

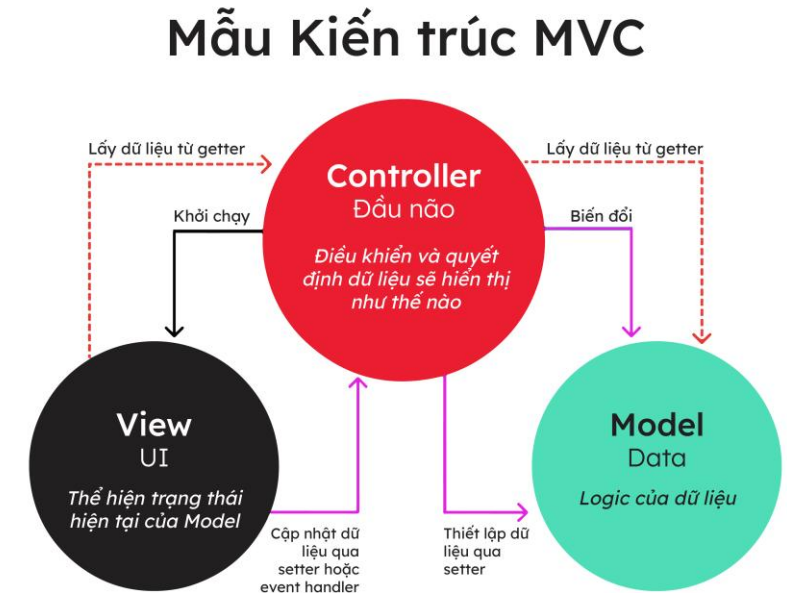
Modernize

Minimal code changes

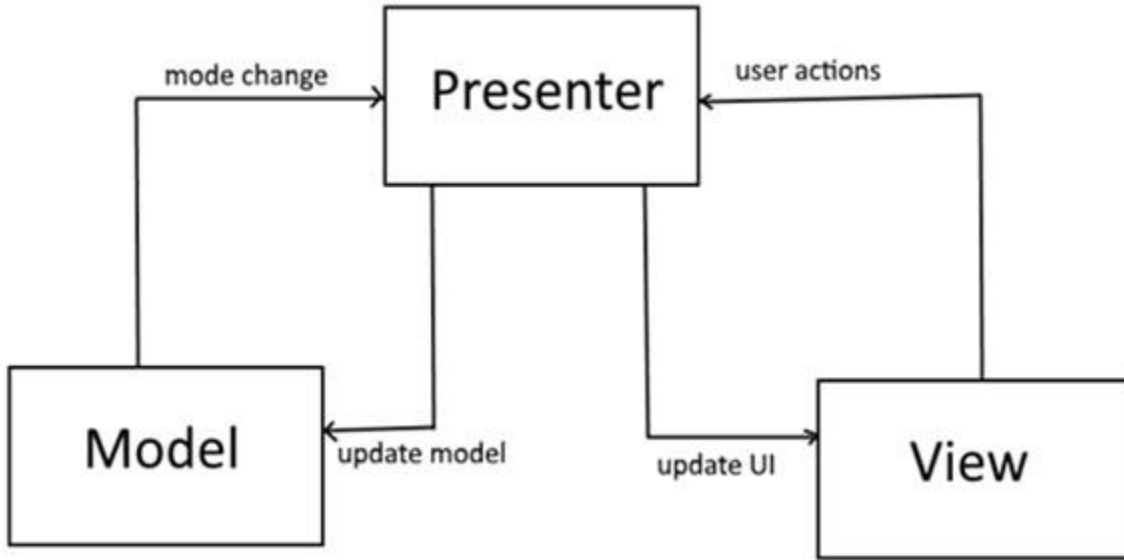
Architected for the cloud, needs new code

| 7.3 MÔ HÌNH LẬP TRÌNH MVC

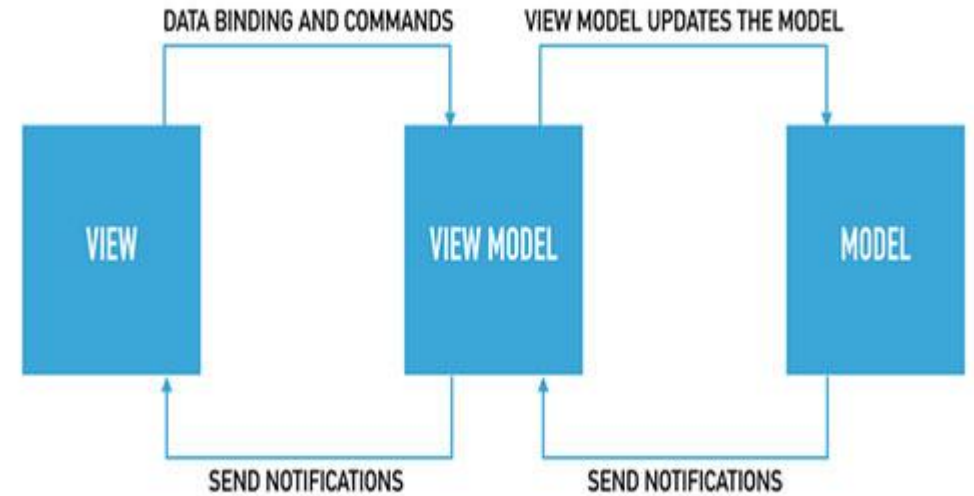
- MVC là viết tắt của Model-View-Controller. Cấu trúc MVC là một mẫu kiến trúc/mẫu thiết kế tách ứng dụng thành ba thành phần logic chính: Model, View và Controller. Mỗi thành phần kiến trúc được xây dựng để xử lý các khía cạnh phát triển cụ thể của một ứng dụng.
- **Ưu điểm:**
 - Dễ dàng bảo trì mã, dễ dàng mở rộng và phát triển.
 - Các thành phần độc lập với người dùng.
 - Đơn giản hóa bằng cách chia web thành ba phần.
 - Chỉ sử dụng Controller để xử lý các yêu cầu của web thông qua một bộ điều khiển duy nhất.
 - Thân thiện với công cụ tìm kiếm.
 - Tương đối nhẹ và tiết kiệm băng thông.
- **Nhược điểm:**
 - Code tương đối phức tạp.
 - Đôi lúc điều hướng khung có thể phức tạp.
 - Tính linh hoạt chưa cao.



| Một số mô hình MVC khác

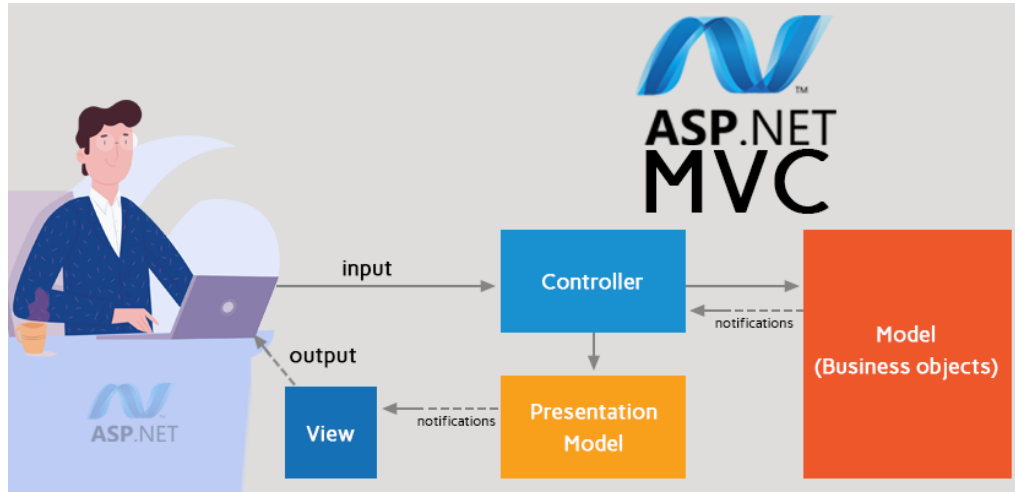


Model, View, Presenter (MVP)



Model, View, ViewModel (MVVM)

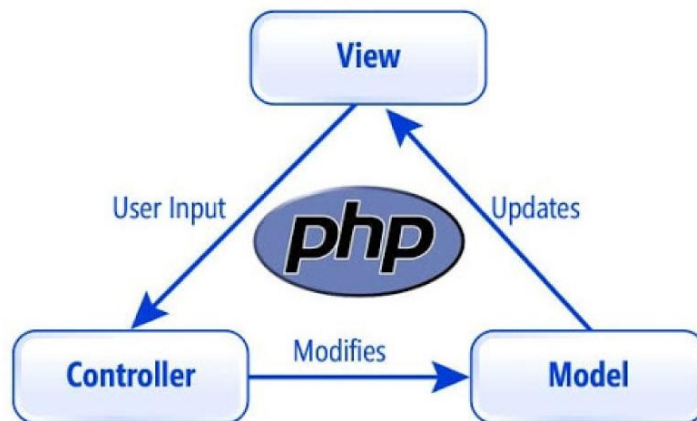
| Công nghệ hỗ trợ lập trình MVC



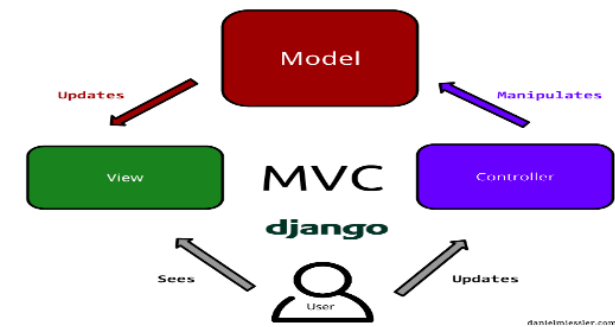
ASP.NET MVC



Struts Framework (Java)



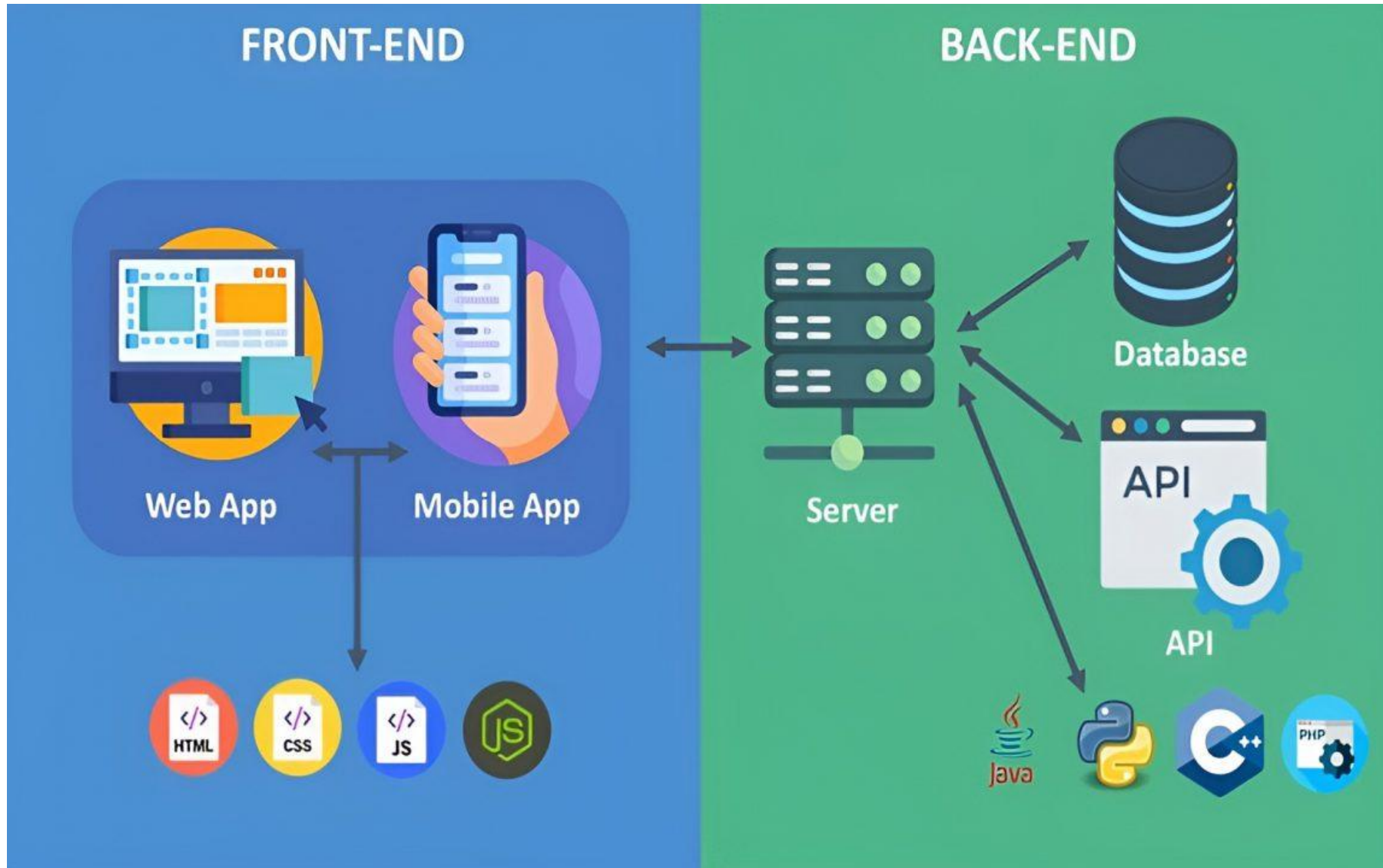
PHP



Model View Controller(MVC) in Django

Django (Python)

| 7.4 LẬP TRÌNH FRONT END VÀ BACK END



| Front End và Back End

- **Front End** là phần giao diện người dùng của một trang web hoặc ứng dụng. Đây là phần mà người dùng trực tiếp nhìn thấy và tương tác khi truy cập website hoặc sử dụng ứng dụng.
- **Lập trình Front End** là việc sử dụng các ngôn ngữ **HTML**, **CSS** hay ngôn ngữ lập trình **Javascript** để các lập trình viên thiết kế ra các **giao diện ứng dụng** hoặc **trang web cho người dùng**.
- **Back End**, hay còn gọi là phía máy chủ, là phần cốt lõi của một website hoặc ứng dụng mà người dùng không trực tiếp nhìn thấy được.
- **Lập trình Back End** là tạo ra những dòng code giúp trình duyệt giao tiếp với cơ sở dữ liệu để lưu trữ, đọc, cập nhật hoặc xóa thông tin từ cơ sở dữ liệu.
 - Sử dụng các ngôn ngữ phía máy chủ như C#, Java, PHP, Python, Ruby ...

| Front End và Back End



Frontend

Focuses on layout, animations, content organization, navigation, graphics.

Programming languages:
JavaScript, HTML, CSS



Backend

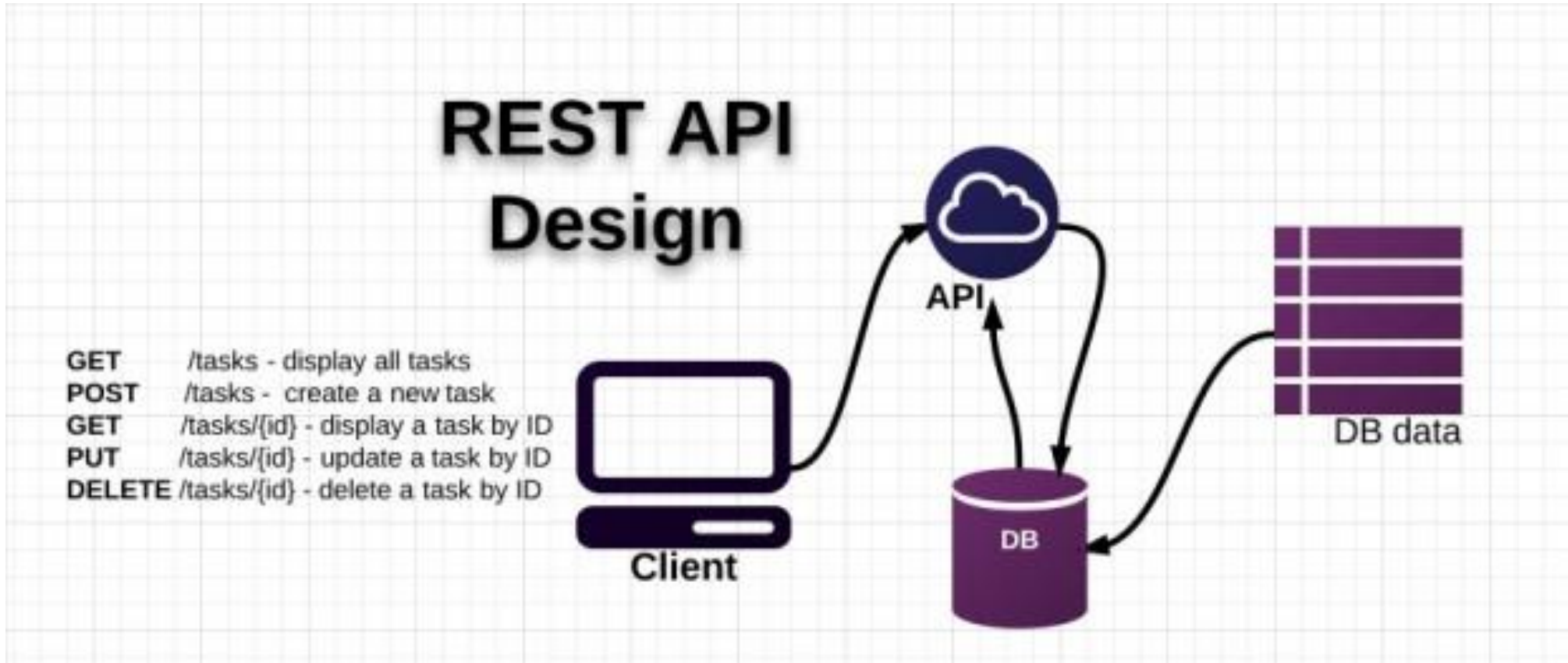
Focuses on building code, debugging, database management.

Programming languages:
Node.js, Python, Java

| RESTful API là gì?

- **RESTful API** (Representational State Transfer API) là một kiểu thiết kế và kiến trúc phần mềm được sử dụng để xây dựng các dịch vụ web (web services) có khả năng trao đổi dữ liệu và thực hiện các thao tác thông qua giao thức HTTP (HyperText Transfer Protocol).
- **RESTful API** dựa trên các nguyên tắc của kiến trúc **REST**, một phong cách kiến trúc phần mềm được giới thiệu bởi Roy Fielding trong luận văn tiến sĩ của ông vào năm 2000. RESTful API xác định các tài nguyên là các đối tượng có thể truy cập thông qua các URL và thực hiện các thao tác CRUD (**Create, Read, Update, Delete**) trên tài nguyên đó bằng cách sử dụng các phương thức HTTP như GET, POST, PUT, DELETE.
- **Các phương thức:**
 - GET: Trả về một Resource hoặc một danh sách Resource.
 - POST: Tạo mới một Resource.
 - PUT: Cập nhật thông tin cho Resource (toàn bộ resource).
 - PATCH: Cập nhật thông tin cho resource (một phần resource).
 - DELETE: Xoá một Resource.
- Dữ liệu thường được trả về ở dạng JSON.

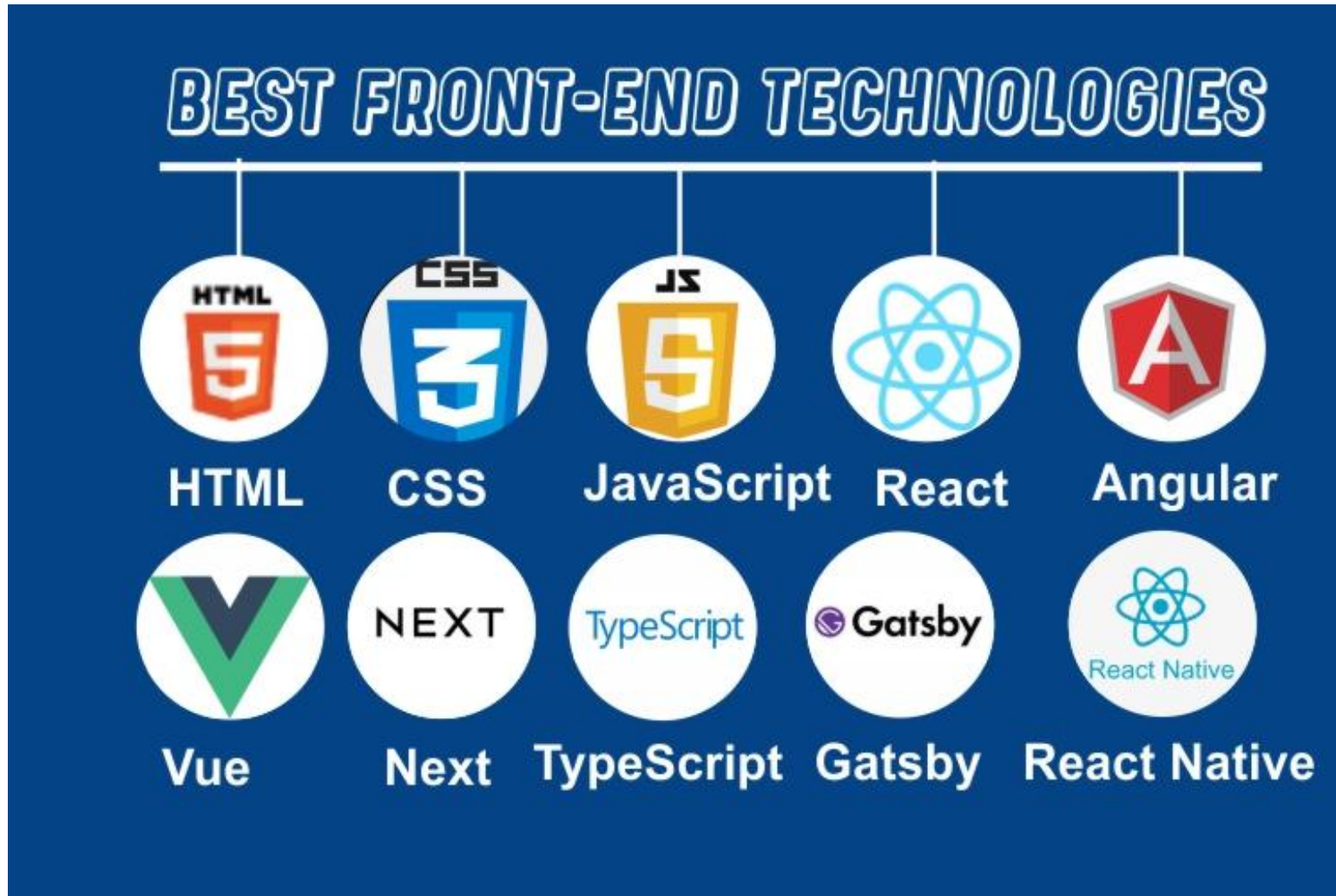
| RESTful API là gì?



| Một số lưu ý khi xây dựng RESTful API

- Sử dụng HTTP Header để chuẩn hóa lại định dạng request và response (ví dụ Content-Type).
- Xây dựng version cho api (ví dụ /api/v1).
- Sử dụng JSON Web Tokens (JWT) để tăng cường bảo mật, xác thực API thông qua token.
- Sử dụng HTTP status code phù hợp (200 OK, 400 Bad Request...)
 - 1xx: information.
 - 2xx: success.
 - 3xx: redirection.
 - 4xx: client error.
 - 5xx: server error.
- Xây dựng convention chung:
 - Thống nhất quy tắc đặt tên API.
 - Đừng dùng underscore (_), hãy dùng hyphen (-) trong URI.
 - URI đều là chữ viết thường (lowercase).
 - Đừng nên sử dụng đuôi file (extension) trong URI (VD: .html, .xml, .json)
- Các API nên chạy ở chế độ HTTPS thay vì HTTP để mã hóa dữ liệu truyền nhận.

| Các công nghệ Front End tốt nhất năm 2024



| React.js

- **React.js (React)** là một framework front end phổ biến được phát triển bởi Facebook. Với khả năng cập nhật giao diện người dùng nhanh mà không gây ảnh hưởng hiệu suất, React.js đang cách mạng hóa cách xây dựng giao diện người dùng tương tác cho các ứng dụng web.
- Tính năng chính của React là virtual **Document Object Model (DOM)** được ca ngợi vì hiệu suất vượt trội và được coi là một trong những framework dễ học nhất.
- Front end này là lựa chọn phù hợp nếu bạn muốn tiết kiệm thời gian trong việc phát triển giao diện tương tác vì các thành phần của React có thể sử dụng lại được.

The screenshot shows the React.js website with a dark header containing the React logo, navigation links (Docs, Tutorial, Blog, Community), a search bar, and version information (v17.0.1, Languages, GitHub). The main content area features two examples of components.

An Application

Using `props` and `state`, we can put together a small Todo application. This example uses `state` to track the current list of items as well as the text that the user has entered. Although event handlers appear to be rendered inline, they will be collected and implemented using event delegation.

LIVE JSX EDITOR (JSX? checkbox checked)

```
class TodoApp extends React.Component {
  constructor(props) {
    super(props);
    this.state = { items: [], text: '' };
    this.handleChange = this.handleChange.bind(this);
    this.handleSubmit = this.handleSubmit.bind(this);
  }

  render() {
    return (
      <div>
        <h3>TODO</h3>
        <TodoList items={this.state.items} />
        <Form onSubmit={this.handleSubmit}>
          <label htmlFor="new-todo">
            What needs to be done?
          </label>
          <input type="text" value={this.state.text} onChange={this.handleChange} />
          <button type="button" value="Add #1" />
        </Form>
      </div>
    );
  }
}
```

RESULT

TODO

What needs to be done?

Add #1

A Component Using External Plugins

React allows you to interface with other libraries and frameworks. This example uses `remarkable`, an external Markdown library, to convert the

LIVE JSX EDITOR (JSX? checkbox checked)

```
class MarkdownEditor extends React.Component {
  constructor(props) {
    super(props);
    this.md = new Remarkable();
    this.handleChange = this.handleChange.bind(this);
    this.state = { value: 'Hello, **world**!' };
  }

  render() {
    return (
      <div>
        <input type="text" value={this.state.value} onChange={this.handleChange} />
        <div>
          Enter some markdown
          <div>
            Hello, **world**!
          </div>
        </div>
      </div>
    );
  }
}
```

RESULT

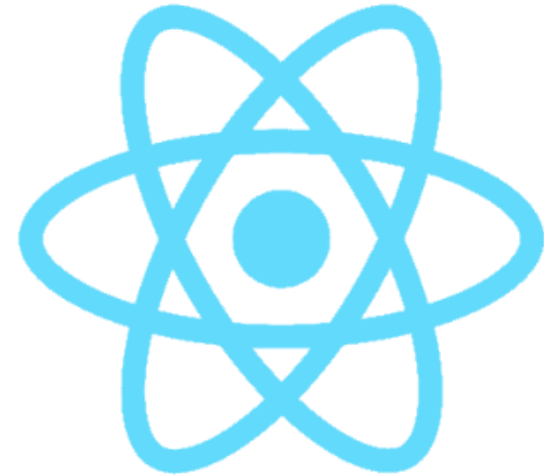
Input

Enter some markdown

Hello, **world**!

| React

- **Ưu điểm:**
 - Được hỗ trợ bởi Facebook.
 - Được cập nhật thường xuyên.
 - Virtual DOM cho phép hoạt động tốc độ cao.
 - Có thể kết hợp với nhiều thư viện JS khác.
 - Dễ dàng di chuyển giữa các phiên bản khác nhau.
 - Phù hợp cho người mới bắt đầu.
- **Nhược điểm:**
 - Thiếu tài liệu được biên soạn kỹ lưỡng.
 - Phức tạp trong việc học cú pháp JSX.



| Angular.js

- **Angular.js** (Angular) là một framework Front end được phát triển bởi Google. Framework này sử dụng mã nguồn mở dựa trên **TypeScript** để xây dựng các ứng dụng web.
- Các tính năng chính của Angular là: Liên kết dữ liệu 2 chiều; Tích hợp TypeScript; Cấu trúc dựa trên MVC.
- Angular thường được các nhà phát triển web sử dụng để phát triển các ứng dụng phức tạp, có quy mô lớn, yêu cầu khả năng tương tác tốt, phản hồi nhanh và đa dạng tính năng.
- **Ưu điểm:**
 - Code dễ dàng hơn nhờ khả năng tái cấu trúc và điều hướng nâng cao.
 - Kiến trúc dựa trên thành phần.
 - Hệ sinh thái lớn, được Google hỗ trợ.
 - Angular Material giúp đơn giản hóa quá trình xây dựng giao diện theo Material Design.
 - Hiệu suất cao.
- **Nhược điểm:**
 - Codebase nặng.
 - Khó debug hơn so với các Framework JavaScript khác.
 - SEO hạn chế.



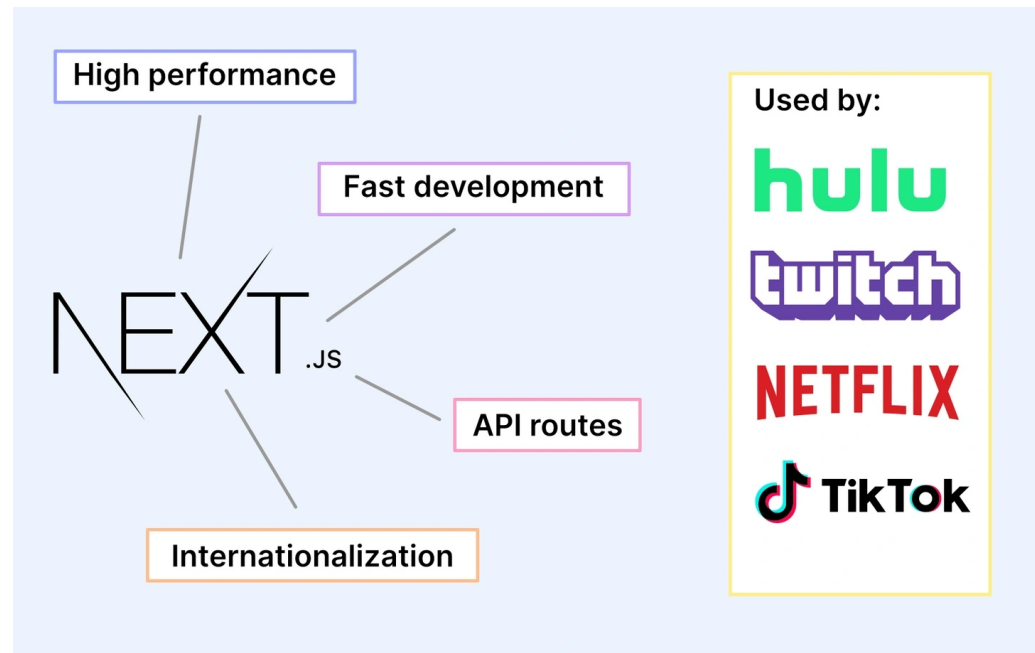
| Vue.js

- **Vue.js** (Vue) là một framework front end được phát triển bởi Evan You. Đây là thư viện JavaScript nhẹ và dễ học về xây dựng giao diện người dùng. Các tính năng chính của Vue bao gồm các liên kết dữ liệu phản ứng và tổng hợp, virtual DOM và kiến trúc dựa trên thành phần.
- Vue phù hợp để xây dựng các ứng dụng vừa và nhỏ, bao gồm blog, diễn đàn và trang web cá nhân.
- **Ưu điểm:**
 - Cú pháp đơn giản, dễ hiểu.
 - Kích thước nhỏ gọn, tải nhanh và sử dụng ít tài nguyên hệ thống.
 - Có thể sử dụng với các thư viện và Framework JavaScript khác.
- **Nhược điểm:**
 - Còn non trẻ và thiếu một số thư viện và công cụ.
 - Hạn chế đối với các dự án lớn.
 - Việc tối ưu hóa SEO cho các ứng dụng Vue.js có thể gặp khó khăn.



| NEXTJS

- **NextJS** là một framework có mã nguồn mở được xây dựng trên nền tảng của **React**, cho phép chúng ta xây dựng các trang web tĩnh có tốc độ siêu nhanh và thân thiện với người dùng, cũng như xây dựng các ứng dụng web React.
- **NextJS** được ra đời vào năm 2016, thuộc sở hữu của Vercel. NextJS bắt đầu trở nên phổ biến vào năm 2018 và tiếp tục tăng trưởng mạnh mẽ trong cộng đồng phát triển web. Sự kết hợp của các tính năng như **Server-side Rendering** (SSR) với **Static Site Generation** (SSG) đã giúp NextJS trở thành sự lựa chọn hấp dẫn cho nhiều dự án.



NEXT.js

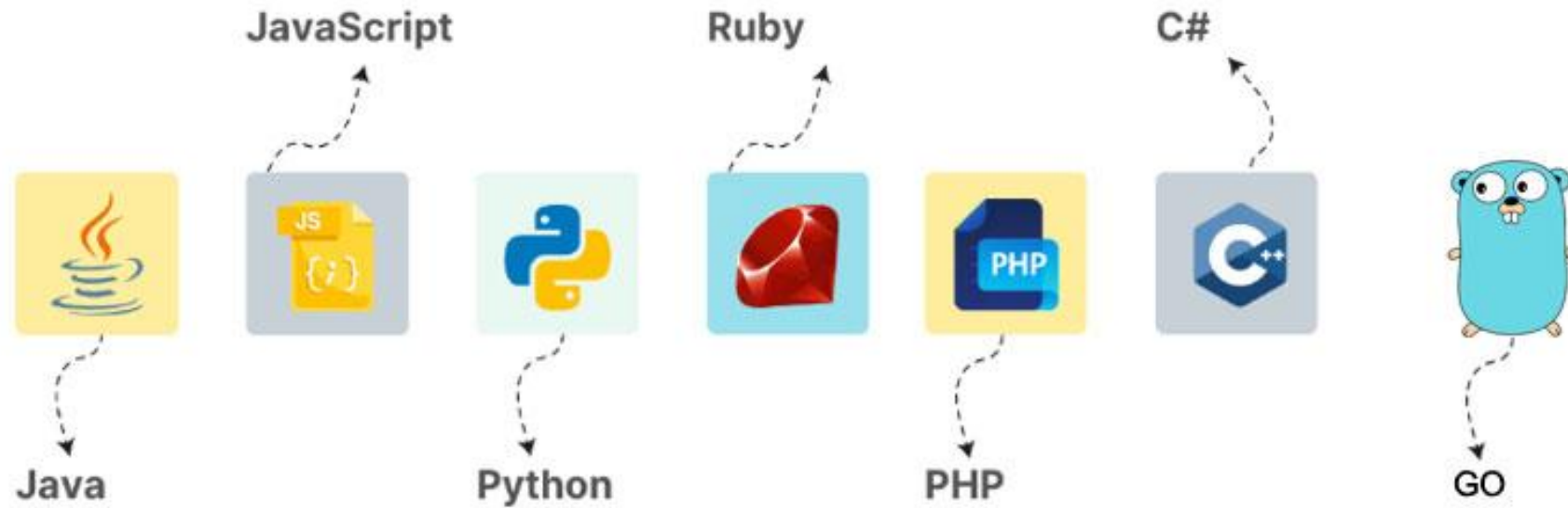
- **Ưu điểm:**

- Sử dụng SSR và SSG: Giúp cải thiện tốc độ tải trang và khả năng SEO.
- Có nhiều tính năng giúp tối ưu hoá hiệu suất.
- Tự động làm mới giao diện mà không cần load lại toàn bộ trang.
- Tự động tạo file CSS dành riêng cho mỗi trang, giúp tránh xung đột.
- Hỗ trợ TypeScript.
- Cộng đồng lớn.
- Hệ sinh thái mạnh mẽ.
- Tích hợp tốt với React.

- **Nhược điểm:**

- Khó học cho người mới.
- Khó khăn trong việc tích hợp với một số thư viện bên ngoài.

| Ngôn ngữ lập trình Back End



| Ngôn ngữ lập trình Back End

Ngôn ngữ	Mô tả	Framework
Java	Mạnh mẽ, bảo mật cao, khả năng mở rộng tốt, cộng đồng lớn	Spring Boot, Hibernate, Struts.
C#	Mạnh mẽ, hiệu suất cao, bảo mật tốt, phát triển bởi Microsoft	ASP.NET (Core) Web API, Entity Framework (Core)
Python	Dễ học, cú pháp đơn giản, đa dạng thư viện, cộng đồng lớn	Django, Flask, FastAPI
JavaScript	Linh hoạt, cộng đồng lớn, nhiều framework, có thể sử dụng cho cả frontend và backend	Node.js, Express.js, NestJS
PHP	Dễ học, dễ sử dụng, phù hợp cho website nhỏ, chi phí thấp	Laravel, Slim Framework
Ruby	Cú pháp đơn giản, dễ học	Rails Framework
Go	Hiệu suất cao, bảo mật tốt, đơn giản, dễ học	Gin, Echo, Beego

I 7.5 LỰA CHỌN CƠ SỞ DỮ LIỆU

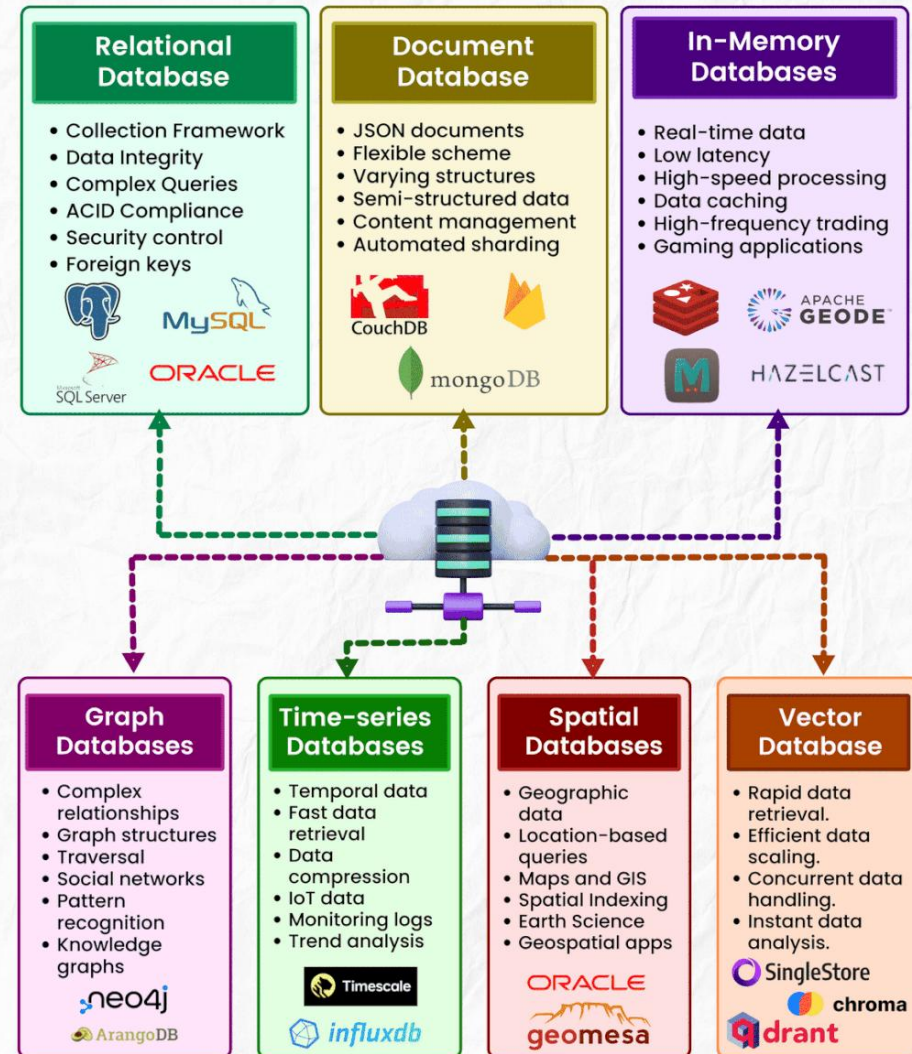
Việc lựa chọn loại cơ sở dữ liệu **quyết định sự thành công cho dự án**, nó phụ thuộc vào:

- Loại dữ liệu: có cấu trúc hay không có cấu trúc.
 - CSDL có cấu trúc: Oracle, MS SQL, MySQL, MariaDB, SQLite...
 - CSDL phi cấu trúc: MongoDB, Cassandra, Redis, Neo4j ...
- Tính nhất quán và tuân thủ tính ACID (*Atomicity, Consistency, Isolation và Durability*).
- Số lượng người dùng, truy cập đồng thời.
- Hiệu suất của ứng dụng.
- Dung lượng dữ liệu của ứng dụng.
- Hạ tầng triển khai: on premises hay on cloud, yếu hay mạnh.
- Chi phí triển khai.
- Bảo mật, sự hỗ trợ từ nhà cung cấp CSDL.
- Chiến lược backup và khôi phục.

MỘT SỐ GỢI Ý CHỌN LOẠI CƠ SỞ DỮ LIỆU

Types of Databases

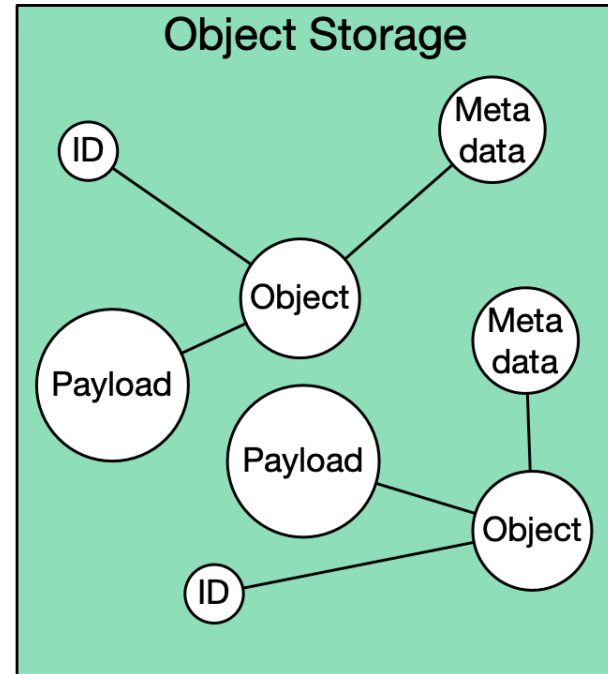
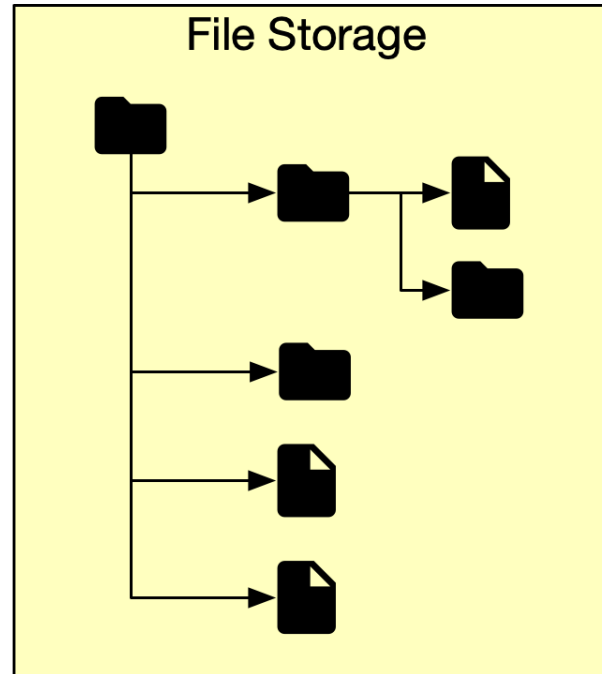
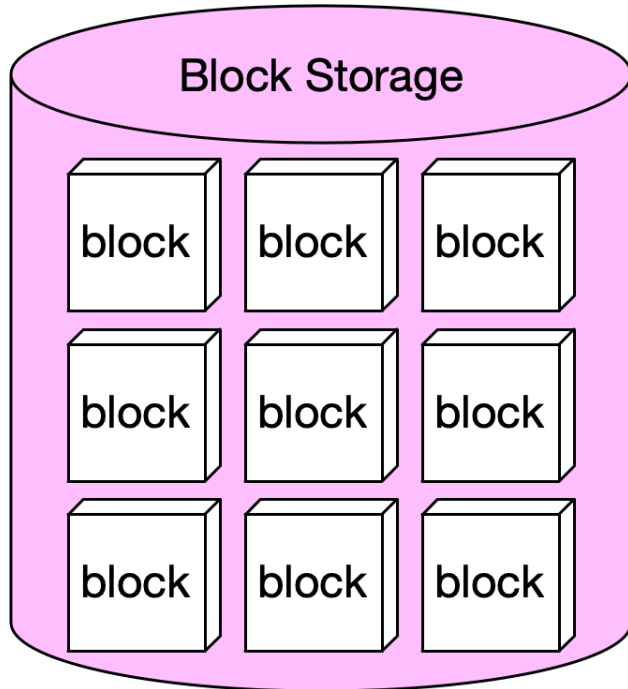
Created by Gina Acosta
Join my newsletter [joinhorizon.ai](#)
and get more free resources!



| MỘT SỐ GỢI Ý LỰA CHỌN CƠ SỞ DỮ LIỆU

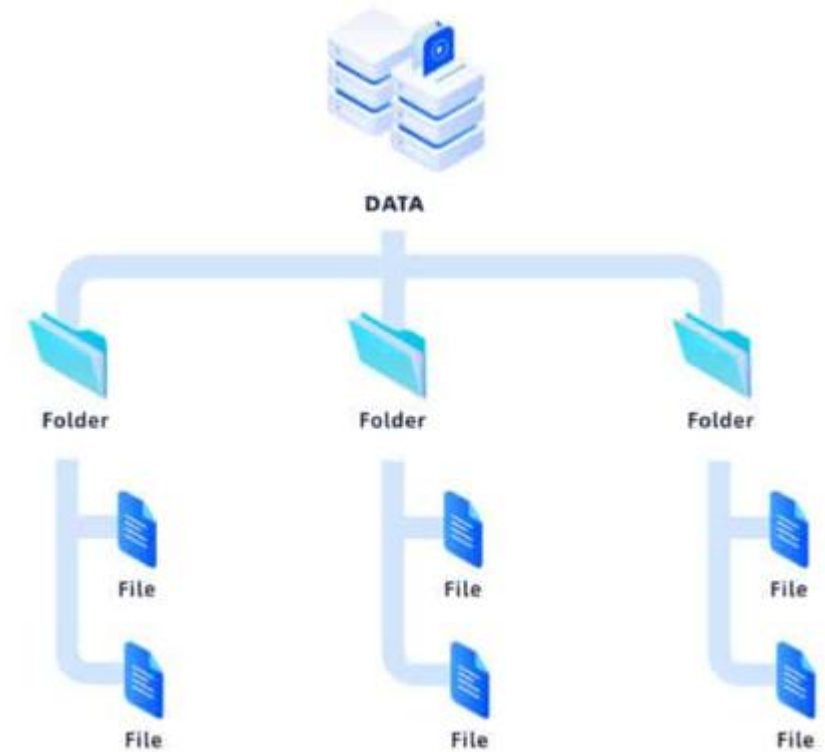
Loại CSDL	Kích thước	Mục đích sử dụng	Thông tin khác
MySQL	Vừa và nhỏ	Website, hệ thống thương mại điện tử, CMS và các ứng dụng dựa trên dữ liệu khác.	Có bản miễn phí và trả phí.
MariaDB	Vừa và nhỏ	Được phát triển từ MySQL nhưng hiệu năng tốt hơn.	Miễn phí.
PostgreSQL	Vừa và nhỏ	Hệ thống kho dữ liệu, dữ liệu không gian địa lý (GIS), ứng dụng web và hệ thống tài chính.	Miễn phí.
MS SQL Server	Vừa và nhỏ	Được sử dụng rộng rãi trong DN, có thể tích hợp vào các sản phẩm khác của Microsoft.	Tốn chi phí.
MongoDB (NoSQL)	Lớn	Phù hợp cho các dữ liệu liên tục thay đổi, phân tích thời gian thực, phân tán, có thể lưu trữ tài liệu JSON. Các ứng dụng phổ biến bao gồm CMS, IoT.	Miễn phí.
Oracle	Lớn	Các hệ thống lớn, hiệu suất cao, bảo mật và ổn định.	Tốn chi phí.
Cassandra (NoSQL)	Lớn	Xử lý dữ liệu lớn, đảm bảo tính khả dụng cao. Sử dụng cho Messaging, IoT, Analytics.	Miễn phí.
Redis	Bộ nhớ trong	Phổ biến trong các ứng dụng yêu cầu độ trễ thấp và real time như gaming, dịch vụ tài chính, chăm sóc sức khỏe và IoT.	Miễn phí.

| 7.6 LỰA CHỌN CÔNG NGHỆ LƯU TRỮ



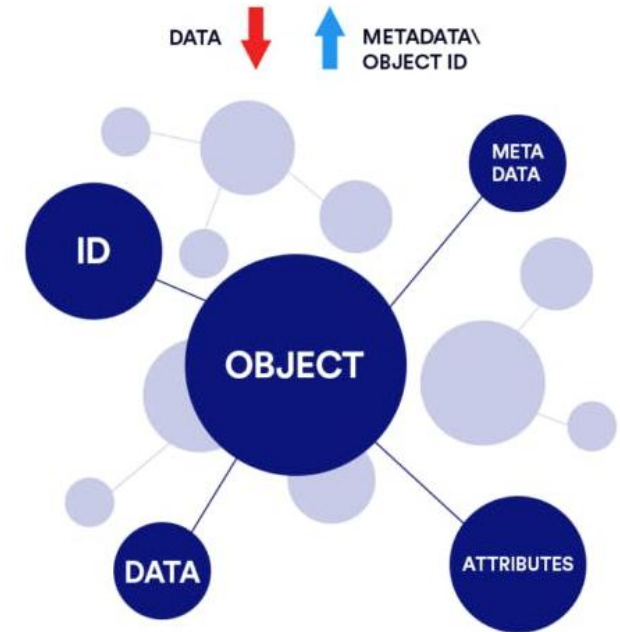
| 7.6.1 File Storage

- **File Storage** là một hệ thống lưu trữ dữ liệu truyền thống được phát triển từ những năm 1960 với sự ra đời của các hệ thống máy tính đầu.
- **File storage** là quá trình lưu trữ dữ liệu trong một thiết bị hoặc hệ thống lưu trữ điện tử như máy tính, ổ cứng, server, đám mây, hay các thiết bị lưu trữ khác. File storage cho phép người dùng lưu trữ các loại tệp tin như văn bản, hình ảnh, âm thanh, video, tài liệu, và các loại dữ liệu khác.
- **File Storage** tổ chức dữ liệu dưới dạng các tệp tin (files) và thư mục (folder), cho phép người dùng dễ dàng lưu trữ, tìm kiếm, truy xuất và quản lý thông tin.
- **Ưu điểm:**
 - Dễ sử dụng và quản lý.
 - Tính tương thích cao.
 - Bảo mật dữ liệu.
 - Tối ưu hóa chi phí.
- **Nhược điểm:**
 - Khả năng mở rộng hạn chế.
 - Lưu trữ trên đám mây sẽ có vấn đề về tốc độ truy cập dữ liệu và bảo mật.



| 7.6.2 Object Storage

- **Object storage** là một phương thức lưu trữ dữ liệu phân tán, trong đó dữ liệu được lưu trữ và quản lý dưới dạng các đối tượng (**object**) độc lập. Mỗi đối tượng chứa **dữ liệu**, **metadata** và một định danh (**identifier**) duy nhất để phân biệt với các đối tượng khác trong hệ thống.
- **Object storage** cung cấp một phương thức lưu trữ phân tán và độc lập với phần cứng, giúp tăng tính sẵn sàng và độ tin cậy của hệ thống lưu trữ. Đồng thời, nó cũng cho phép truy xuất dữ liệu từ bất kỳ đâu thông qua các giao thức như HTTP hoặc HTTPS.
- Hệ thống **object storage** cung cấp một giao diện API để cho phép người dùng tương tác với các đối tượng, tạo mới, sửa đổi, xóa và truy xuất dữ liệu. **Object storage** cũng hỗ trợ nhiều giao thức truy cập như HTTP, HTTPS, S3, Swift, NFS và SMB để đảm bảo tính linh hoạt và tương thích với các ứng dụng khác nhau.
- **Object storage** thường được sử dụng cho các ứng dụng lưu trữ dữ liệu lớn như hình ảnh, video, âm thanh, tài liệu và các ứng dụng trên đám mây.



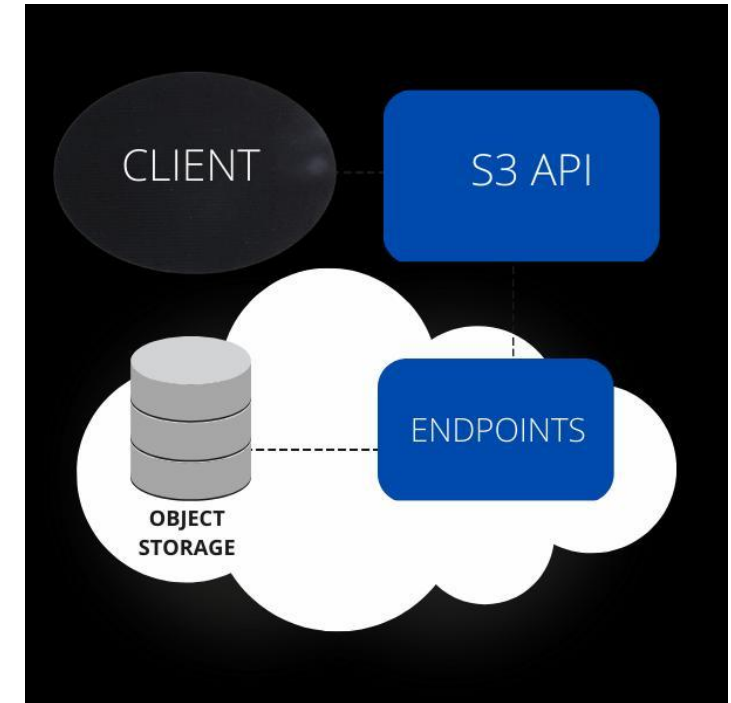
| 7.6.2 Object Storage

- **Ưu điểm:**

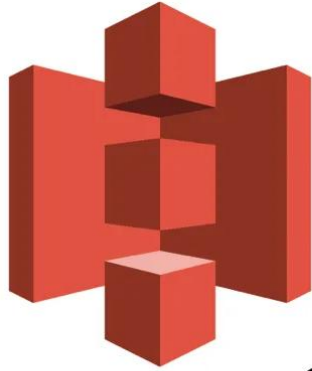
- **Khả năng lưu trữ và quản lý dữ liệu lớn:** cho phép lưu trữ và quản lý dữ liệu lớn một cách hiệu quả, có thể lưu trữ hàng tỷ hoặc thậm chí hàng trăm tỷ đối tượng một cách dễ dàng.
- **Tính toàn vẹn và khả năng phục hồi dữ liệu:** có khả năng tự động tạo ra các bản sao của dữ liệu và phân phối chúng đến các máy chủ khác nhau trong hệ thống.
- **Tính sẵn sàng và độ tin cậy:** được phân tán trên nhiều máy chủ khác nhau, giúp đảm bảo tính sẵn sàng và độ tin cậy của hệ thống lưu trữ.
- **Tính tương thích và tính linh hoạt:** hỗ trợ nhiều giao thức truy cập như HTTP, HTTPS, S3, Swift, NFS và SMB để đảm bảo tính tương thích và tính linh hoạt với các ứng dụng khác nhau.

- **Nhược điểm:**

- Tốc độ truy cập chậm hơn so với block storage hoặc file storage.
- Chi phí cao hơn so với các phương pháp lưu trữ khác.
- Không thích hợp cho các ứng dụng yêu cầu tốc độ truy cập cao và tối ưu hóa cho hiệu suất.
- Khó khăn trong việc xử lý dữ liệu có kích thước lớn.
- Khó khăn trong việc tích hợp và quản lý.
- Có nguy cơ về bảo mật.



| Dùng thử Object Storage ở đâu?



Amazon S3

<https://aws.amazon.com/vi/s3/>

Thông tin gói miễn phí S3 Storage của
Amazon gói Free Tier:

Dung lượng: 5GB

Băng thông (in/out): 100GB



<https://cloudinary.com>

Miễn phí 25GB lưu trữ
25 GB băng thông



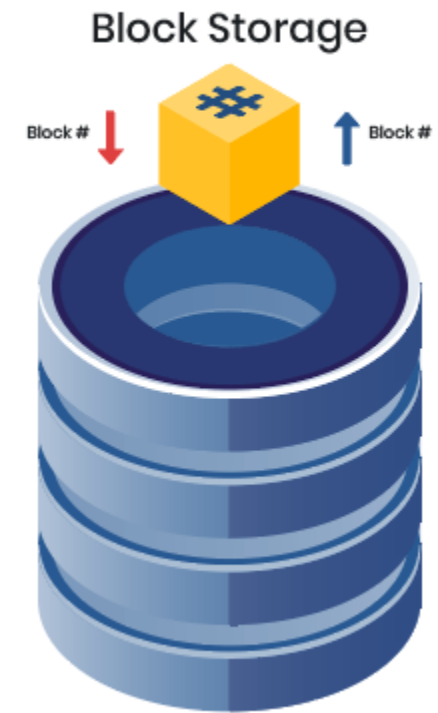
MINIO

<https://min.io>

Mã nguồn mở miễn phí
Tự cài đặt và trải nghiệm

| 7.6.3 Block Storage

- **Block storage** là một dạng lưu trữ dữ liệu trong đó dữ liệu được phân chia thành các khối (block) có kích thước nhất định và được lưu trữ trên các thiết bị lưu trữ như ổ đĩa cứng, ổ đĩa SSD hoặc các thiết bị lưu trữ mạng (SAN). Khối dữ liệu trong **block storage** được truy cập thông qua các địa chỉ duy nhất được gọi là "địa chỉ block", và các khối này có thể được sắp xếp và đọc ghi một cách độc lập nhau.
- **Block storage** thường được sử dụng trong các ứng dụng yêu cầu tốc độ truy xuất dữ liệu nhanh như cơ sở dữ liệu hoặc các ứng dụng yêu cầu lưu trữ dữ liệu lớn và truy xuất dữ liệu dưới dạng ngẫu nhiên như các ứng dụng máy chủ.
- **Ưu điểm:**
 - Tốc độ truy xuất nhanh.
 - Linh hoạt trong việc quản lý dữ liệu.
 - Độ tin cậy cao.
 - Khả năng mở rộng dễ dàng.
- **Nhược điểm:**
 - Giá thành cao.
 - Khả năng phức tạp trong triển khai và quản lý.
 - Giới hạn trong việc chia sẻ và cộng tác.
 - Khả năng phục hồi và mở rộng hạn chế trong một số trường hợp.

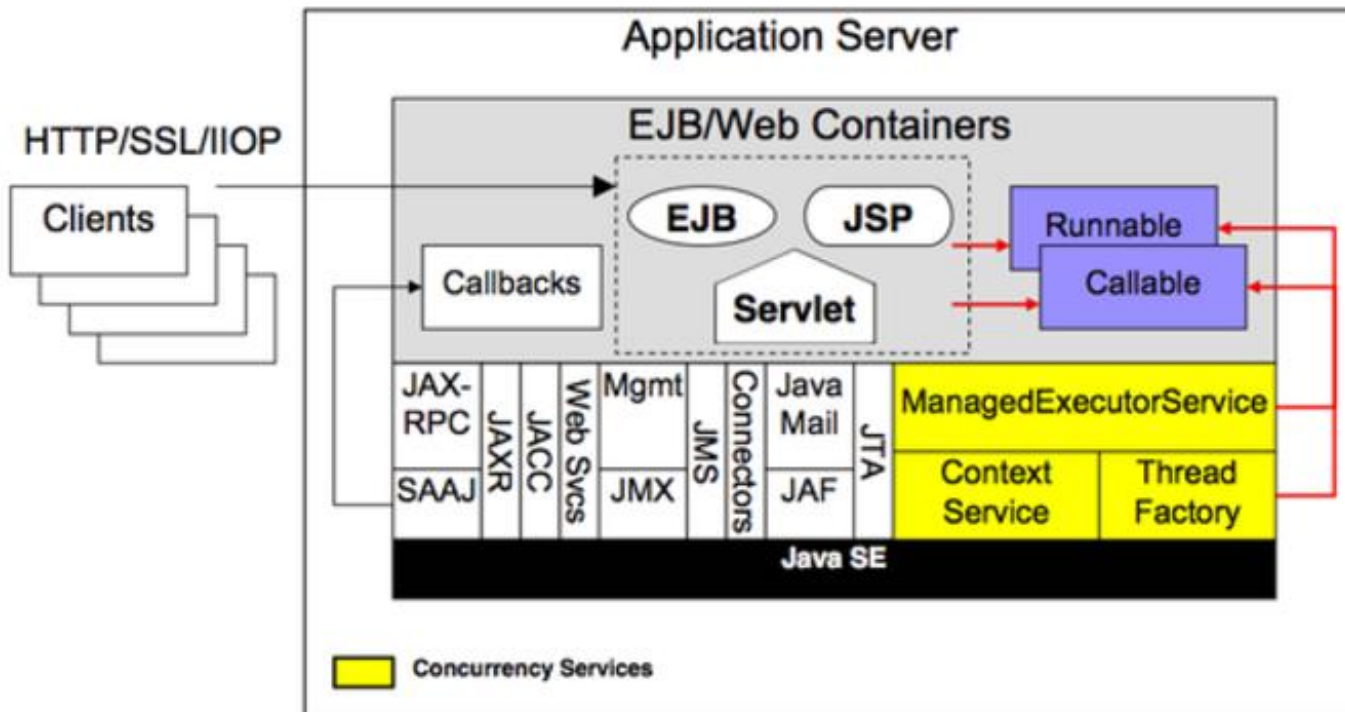


| 7.7 CÁC CÔNG NGHỆ KHÁC

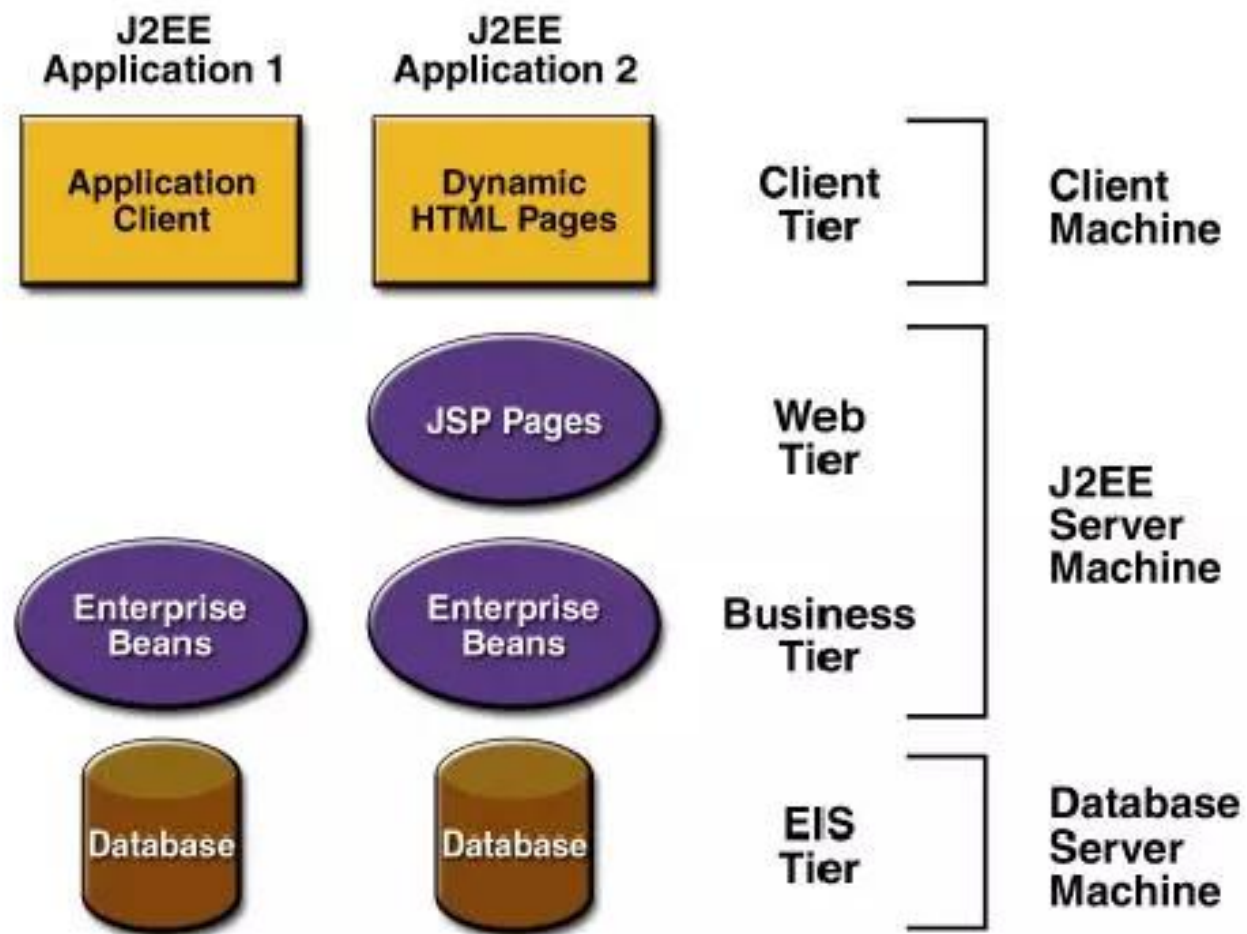
1. J2EE.
2. Xây dựng ứng dụng Cross Platform.
3. Triển khai ứng dụng với Docker.

| 7.7.1 J2EE

- **J2EE** tên đầy đủ là **Java 2 Enterprise Edition** dịch sang tiếng việt có nghĩa là là "Java Phiên Bản Doanh Nghiệp", nó là một nền tảng dành cho việc xây dựng các ứng dụng cấp doanh nghiệp.
- **J2EE** giúp cho việc lập trình ứng dụng được đơn giản hóa và giảm thiểu lập trình bằng cách tạo các thành phần module, có thể tái sử dụng và tiêu chuẩn hóa.



| Cấu trúc của J2EE

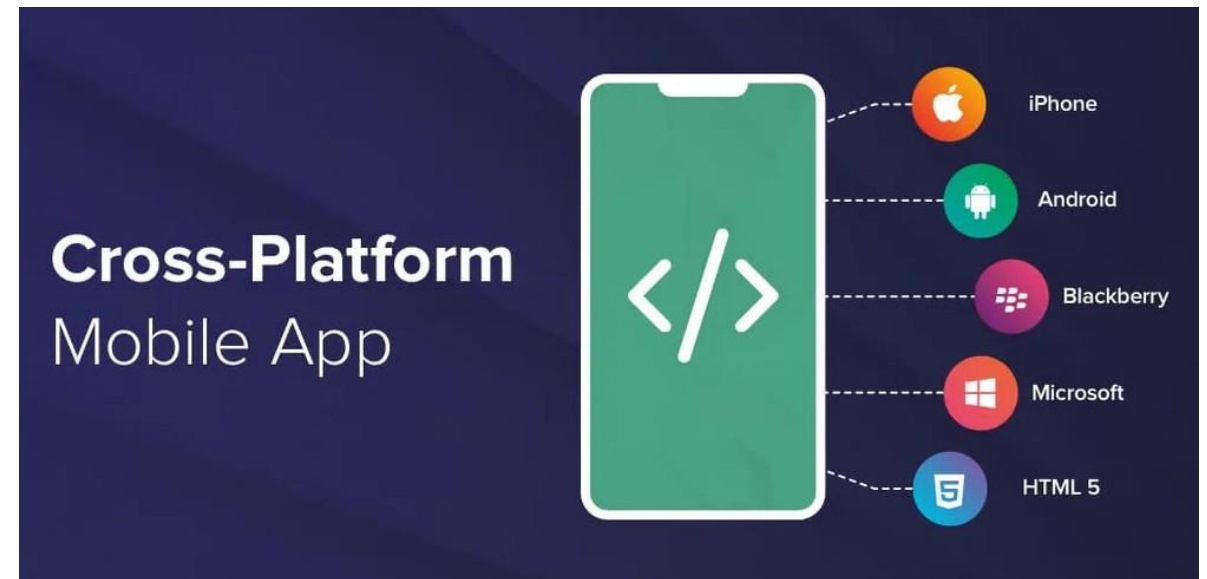


| Các thành phần của J2EE

- **J2EE** được xây dựng trên một mô hình container component. Bốn container component cốt lõi cung cấp môi trường cho các component khác của J2EE thông qua các API. Những component cốt lõi này liên quan đến bốn kiểu container được hỗ trợ trong J2EE bao gồm, **Application Client, Applet, Web và EJB**.
 - **Java Application** – component này là 1 chương trình standalone chạy bên trong Application Client container.
 - **Applet** – Applet component là java applet chạy bên trong Applet container, chính là web browser có hỗ trợ công nghệ Java.
 - **Servlet và JSP** – đây là Web-based component chạy ở bên trong Web container, được hỗ trợ bởi Web Server. Web container là một môi trường run-time cho servlet và jsp.
 - **Enterprise JavaBean (EJB)** – EJB component là business component chạy bên trong EJB container. EJB component là phần nhân, cốt lõi của ứng dụng J2EE. EJB container cung cấp các dịch vụ quản lý transaction, bảo mật, quản lý trạng thái, quay vòng tài nguyên (resource pooling).

| 7.7.2 Xây dựng ứng dụng Cross Platform

- **Cross Platform** là một thuật ngữ để chỉ việc xây dựng một ứng dụng duy nhất có thể chạy trên nhiều nền tảng hệ điều hành khác nhau, thay vì phát triển các phiên bản ứng dụng riêng cho từng nền tảng. Hiểu đơn giản hơn là chỉ với một bản code, bạn có thể biên dịch và sử dụng nó trên đa nền tảng ứng dụng, giúp bạn tiết kiệm được thời gian và công sức đáng kể.
- **Các lợi ích mang lại:**
 - Kiến trúc mạch lạc: tuân thủ MVC hoặc MVVM.
 - Ít code: viết 1 lần, chạy nhiều nền tảng.
 - Dễ bảo trì và mở rộng.
 - Tính tái sử dụng cao.
- **Ưu điểm:**
 - Phát triển ứng dụng nhanh hơn.
 - Tiếp cận thị trường nhanh.
 - Giảm chi phí.
- **Nhược điểm:**
 - Trải nghiệm người dùng thấp.
 - Hiệu suất có thể thấp hơn Native app.

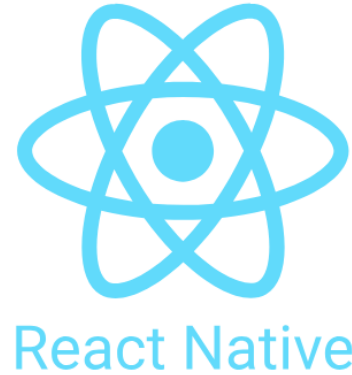


| Phân biệt Cross Platform với Native App, Hybrid App

- **Native App** là một chương trình, phần mềm ứng dụng được phát triển riêng cho chỉ một nền tảng hoặc một thiết bị cụ thể và hệ điều hành của nó (Android/iOS).
- **Hybrid App** là ứng dụng “pha trộn” cả giải pháp Native và Web, được viết bằng các ngôn ngữ phát triển web (CSS, HTML và JavaScript) giúp lập trình viên tạo khung rỗng và đưa lên kho ứng dụng.
- **Cross Platform** là ứng dụng đa nền tảng và được sử dụng phù hợp cho nhiều loại nền tảng và loại thiết bị khác nhau. Nó được xem là giải pháp giúp khắc phục các vấn đề của Native App.



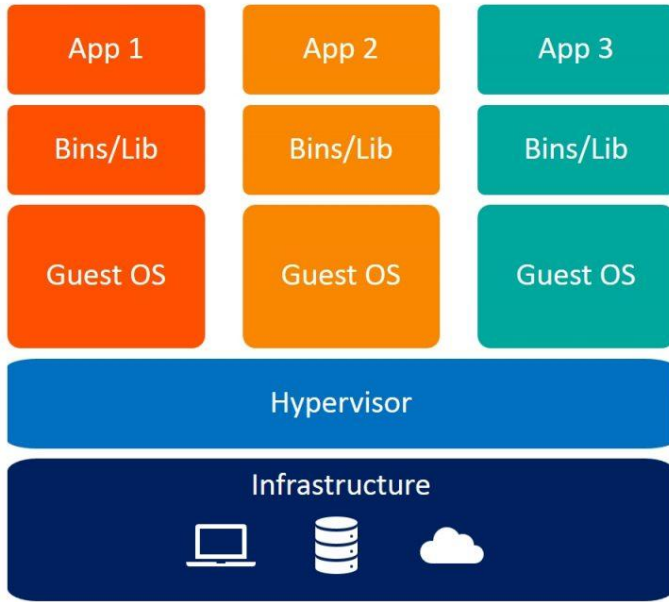
| Một số công nghệ xây dựng ứng dụng Cross Platform



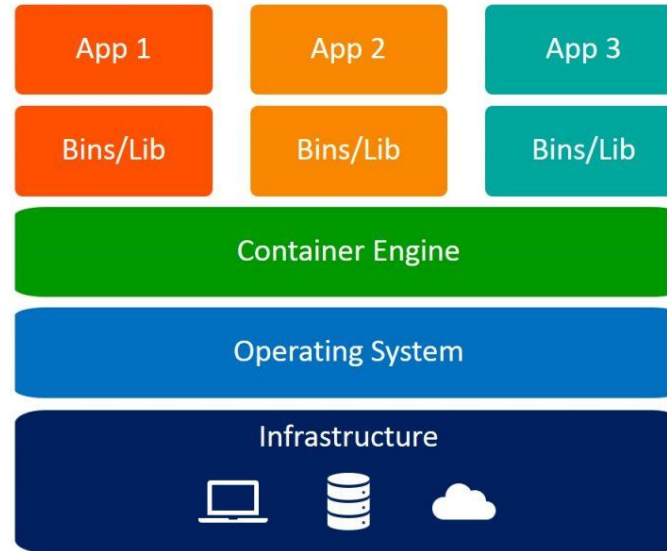
| 7.7.3 Triển khai ứng dụng docker

- **Docker** là một hệ thống chuyên cung cấp phương pháp để dựng, kiểm thử hay triển khai những ứng dụng được thuận tiện hơn bằng cách sử dụng các cấu trúc containers (trong nền tảng ảo hóa).
- Các cấu trúc **containers** cho phép người lập trình viên tổng hợp vào trong một ứng dụng những tất cả các phần mềm cần thiết, chẳng hạn như là thư viện hay một số các phần mềm phụ thuộc khác, và gói tất cả lại ở dưới dạng một package.
- Tại sao nên dùng Docker?
 - **Tính dễ ứng dụng**: Docker rất dễ cho mọi người sử dụng từ lập trình viên, sys admin... nó tận dụng lợi thế của container để build, test nhanh chóng. Có thể đóng gói ứng dụng trên laptop của họ và chạy trên public cloud, private cloud... Câu thần chú là “Build once, run anywhere”.
 - **Tốc độ**: Docker container rất nhẹ và nhanh, bạn có thể tạo và chạy docker container trong vài giây.
 - **Môi trường chạy và khả năng mở rộng**: Bạn có thể chia nhỏ những chức năng của ứng dụng thành các container riêng lẻ.
 - **Hiệu suất cao**: Docker containers chia sẻ cùng một hệ điều hành kernel, giúp giảm thiểu tài nguyên hệ thống so với các máy ảo (VMs), dẫn đến hiệu suất cao hơn và chi phí thấp hơn.
 - **Quản lý phụ thuộc dễ dàng**: Docker giúp quản lý tất cả các phụ thuộc của ứng dụng trong một container, đảm bảo rằng không có sự mâu thuẫn phiên bản giữa các thư viện và công cụ.

| So sánh sự khác nhau giữa Máy ảo và Docker Container



Virtual Machines

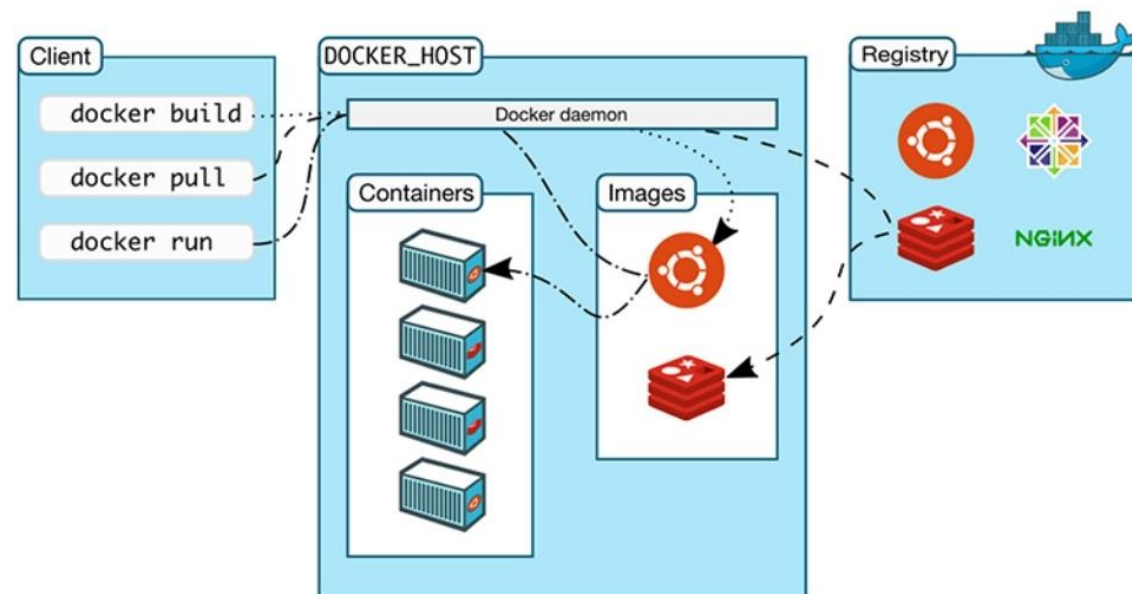


Containers

Máy ảo	Docker container
Kích thước (dung lượng) lớn.	Kích thước (dung lượng) nhỏ.
Hiệu suất hạn chế.	Hiệu suất gốc (native).
Mỗi máy ảo sẽ có một hệ điều hành riêng.	Container sẽ sử dụng hệ điều hành của host.
Ảo hóa về mặt phần cứng	Ảo hóa về mặt hệ điều hành
Thời gian khởi động tính theo phút	Thời gian khởi động tính theo mili giây
Phân bổ bộ nhớ theo nhu cầu cần thiết	Yêu cầu ít dung lượng bộ nhớ hơn
Hoàn toàn bị cô lập và an toàn hơn	Cô lập ở mức tiến trình, có thể kém an toàn hơn

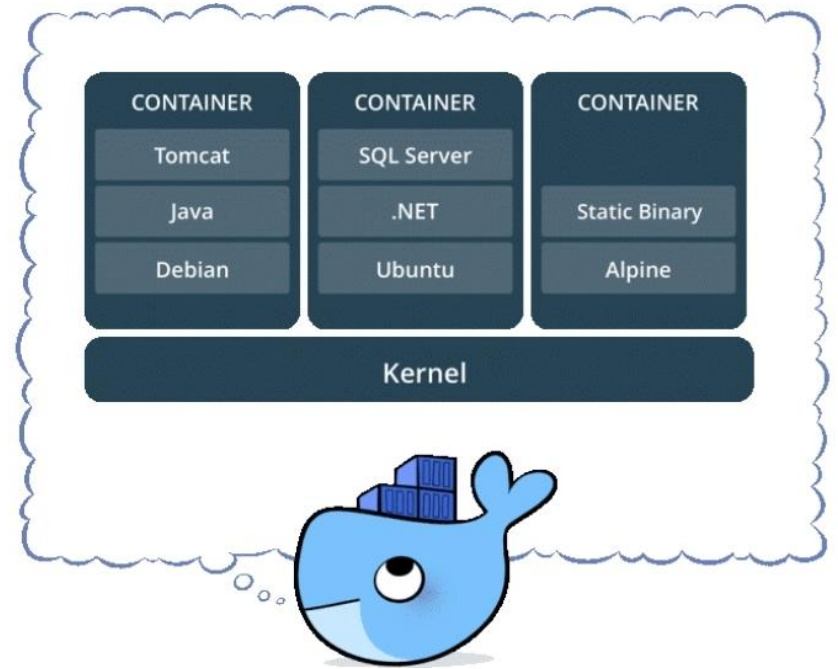
| Các khái niệm cần hiểu về Docker

- **Docker Engine** : là thành phần chính của Docker, đây như là một công cụ dùng để đóng gói ứng dụng (packaged).
- **Docker Hub**: đây là một “github for docker images”. Ở trên Docker Hub có sẽ hàng nghìn public images được tạo ra và chia sẻ bởi cộng đồng.
- **Images**: thuật ngữ này để chỉ một khuôn mẫu để tạo ra một cấu trúc container. Thông thường, image sẽ dựa trên 1 khuôn mẫu đã có sẵn với một số tùy chỉnh thêm.
- **Container**: là một instance của image. Bạn có thể khởi tạo, bắt đầu, tạm dừng, di chuyển hoặc xóa container đi dựa trên giao diện lập trình Docker API hoặc Docker CLI.
- **Docker Client**: là công cụ dành cho người dùng có thể giao tiếp được với Docker host.
- **Docker Daemon**: có thể lắng nghe những yêu cầu từ trên Docker Client, từ đó sẽ quản lý được những đối tượng như là cấu trúc Container, Image, Network hay là Volumes thông qua tiêu chuẩn REST API.
- **Dockerfile**: là một file tổng hợp bao gồm tất cả những chỉ dẫn chi tiết để xây dựng lên một image.
- ...



| Vậy khi nào bạn cần sử dụng Docker?

- Khi bắt đầu triển khai kiến trúc Microservices.
- Khi bạn muốn xây dựng ứng dụng và cần phải scal một cách linh hoạt.
- Khi bạn không muốn tốn nhiều thời gian để config máy local và server cùng một môi trường để chạy được ứng dụng. Bạn chỉ cần build 1 lần chạy ở nhiều nơi mà thôi.
- Sản phẩm của công ty bạn cần một cách tiếp cận mới về xây dựng, đẩy lên server, thực thi ứng dụng một cách nhanh chóng dễ dàng.



| Thử nghiệm Host Docker ở đâu?



Cài đặt lên Localhost:
<https://www.docker.com>

